

Symmetric Encryption: How It Works

Contents

Notation (list of symbols)	1
1 Block ciphers vs. stream ciphers	1
1.1 Symmetric encryption in one line	2
1.2 Block ciphers	2
1.3 Stream ciphers	2
1.4 Side by side	3
1.5 Symbols	3
2 Modes of operation	3
2.1 ECB — and why never to use it	4
2.2 CBC — cipher block chaining	4
2.3 CTR — counter mode	5
2.4 Padding the last block	5
2.5 These modes hide, but do not protect	5
2.6 Symbols	6

Notation (list of symbols)

k	secret key, shared by encryptor and decryptor (symmetric).
P	plaintext: the message to encrypt, a bit string.
C	ciphertext: the encrypted output.
b	block size in bits (e.g. 128 for AES).

1 Block ciphers vs. stream ciphers

A cryptographic hash is *one-way*: you compute a digest and can never recover the message. *Encryption* is the opposite — reversible, with a secret key — and its goal is *confidentiality*: scramble a message so that only a key-holder can read it.

1.1 Symmetric encryption in one line

Symmetric means a single shared secret key k runs both directions:

$$C = \text{Enc}_k(P), \quad P = \text{Dec}_k(C).$$

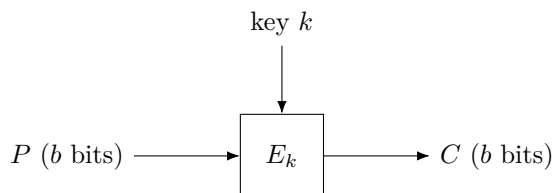
P is the *plaintext* (the message, a bit string), C the *ciphertext* (the scrambled output), and k the key. Dec_k is the exact inverse of Enc_k , so decrypting what you encrypted returns the original. Two families build Enc : *block* ciphers and *stream* ciphers.

1.2 Block ciphers

A block cipher encrypts a *fixed-size block* of bits at a time. Formally it is a keyed *permutation* — a reversible shuffle —

$$E_k : \{0, 1\}^b \rightarrow \{0, 1\}^b,$$

where b is the block size (AES uses $b = 128$ bits). “Permutation” means that for a fixed key, E_k is a bijection on the 2^b possible blocks (one-to-one and onto), so it has an inverse $D_k = E_k^{-1}$ that decryption uses. AES, and the retired DES, are block ciphers.



One block is only b bits, so to encrypt a longer message you need a *mode of operation*: a recipe for chaining many block-cipher calls (CBC, CTR, GCM, ...). Modes usually also require *padding* to fill out the final partial block. (The naive mode, ECB, encrypts each block independently and leaks repeated-block patterns — never use it.)

1.3 Stream ciphers

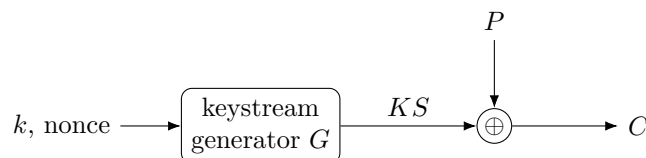
A stream cipher instead generates a long pseudorandom *keystream* from the key and a *nonce*, and combines it with the plaintext one bit (or byte) at a time with XOR:

$$KS = G(k, \text{nonce}), \quad C_i = P_i \oplus KS_i, \quad P_i = C_i \oplus KS_i.$$

The jargon:

- $\oplus$ (XOR, exclusive-or): a bit operation that outputs 1 exactly when its two input bits differ. Its key property is being self-inverse, $x \oplus y \oplus y = x$ — which is why XORing the ciphertext with the *same* keystream gives the plaintext back.
- *keystream* KS : the pseudorandom bit sequence the cipher produces to mask the message.
- *nonce* (“number used once”, a.k.a. IV): a non-secret value that makes each encryption under the same key produce a *different* keystream. It must never repeat for a given key (see the pitfall below).

This is a practical *one-time pad*: the one-time pad XORs the message with a *truly random* pad as long as the message — perfectly secret but impractical (the key is as big as the data and usable once); a stream cipher replaces that true-random pad with a pseudorandom keystream stretched from a short key. ChaCha20 is the modern choice; RC4 is the well-known broken one.



1.4 Side by side

	Block cipher	Stream cipher
Unit	fixed b -bit block	bit / byte at a time
Mechanism	keyed permutation E_k	XOR with a keystream
Long messages	need a mode (CBC, CTR, GCM)	naturally continuous
Padding	usually required	none
Examples	AES, DES (retired)	ChaCha20, RC4 (broken)
Random access	CTR yes, CBC no	yes

The line blurs. Run a block cipher in *counter (CTR)* mode — encrypt a running counter to make a keystream $KS_i = E_k(\text{nonce} \parallel i)$ and XOR it into the plaintext — and the block cipher *becomes* a stream cipher. So “block vs. stream” is partly about the underlying primitive and partly about how you use it.

The one fatal pitfall: never reuse a (key, nonce) pair. If two messages are XORed with the *same* keystream, then

$$C \oplus C' = (P \oplus KS) \oplus (P' \oplus KS) = P \oplus P',$$

the keystream cancels and the attacker learns $P \oplus P'$ — usually enough to unravel both messages. (This is the “two-time pad” break.) A fresh nonce per message avoids it.

1.5 Symbols

New persistent symbols this section — k, P, C, b — are in the global [list of symbols](#). The symbols below are *local to this section*.

Local symbol	Meaning (this section)
$\text{Enc}_k, \text{Dec}_k$	encrypt / decrypt under key k (exact inverses)
E_k, D_k	block-cipher permutation and its inverse, on b -bit blocks
G	keystream generator
KS	keystream (pseudorandom bits); KS_i is its i -th unit
nonce	“number used once”: non-secret, must not repeat per key
$\oplus$	bitwise XOR
P_i, C_i	i -th bit/byte of plaintext / ciphertext
i	position index

2 Modes of operation

A block cipher E_k encrypts exactly one b -bit block (§1). Real messages are longer, shorter, or not a clean multiple of b . A *mode of operation* is the recipe for calling E_k repeatedly to encrypt a

message of any length. Throughout, pad the message (§2.4) and split it into n blocks

$$P_1, P_2, \dots, P_n, \quad \text{each } b \text{ bits,}$$

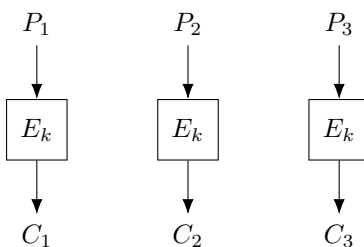
encrypting to ciphertext blocks $C_1, \dots, C_n$. The modes differ in *how* each block is fed through E_k — and that choice decides everything about safety and speed.

2.1 ECB — and why never to use it

The obvious recipe, *Electronic CodeBook* (ECB), encrypts each block on its own:

$$C_i = E_k(P_i), \quad P_i = D_k(C_i),$$

where $D_k = E_k^{-1}$ is the inverse permutation (§1).



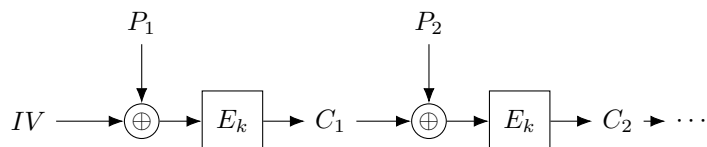
Because E_k is deterministic, *equal plaintext blocks always give equal ciphertext blocks*: if $P_i = P_j$ then $C_i = C_j$. So the ciphertext still shows wherever the message repeats. Encrypt a bitmap image this way and the picture stays recognisable — the infamous “ECB penguin.” ECB leaks structure; never use it for real data. The fix in every mode below is to make encryption *randomised* so identical blocks diverge.

2.2 CBC — cipher block chaining

CBC removes the leak by XORing each plaintext block with the *previous ciphertext block* before encrypting, seeding the chain with a random *initialization vector* IV :

$$C_i = E_k(P_i \oplus C_{i-1}), \quad C_0 \equiv IV.$$

The IV is a non-secret, random b -bit value chosen fresh per message (like the nonce of §1); for CBC it must be *unpredictable*. Because the IV changes, the same plaintext encrypts to different ciphertext each time — the randomisation ECB lacked.



To decrypt, run D_k and undo the XOR. Starting from $C_i = E_k(P_i \oplus C_{i-1})$, apply D_k to both sides: $D_k(C_i) = P_i \oplus C_{i-1}$; then XOR C_{i-1} and use $x \oplus y \oplus y = x$:

$$P_i = D_k(C_i) \oplus C_{i-1}.$$

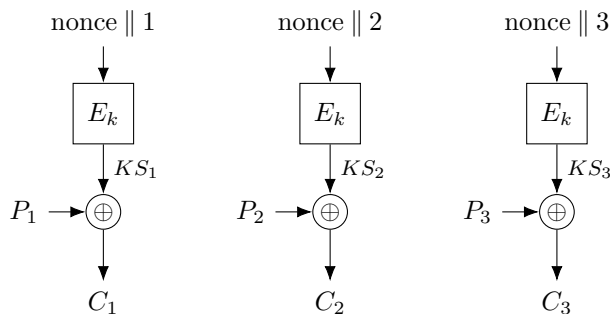
Trade-offs. Encryption is *sequential* — block i needs C_{i-1} , so it cannot be parallelised (decryption can, since all C blocks are already in hand). The message must be padded to a whole number of blocks (below).

2.3 CTR — counter mode

CTR takes the trick from §1: don’t encrypt the plaintext at all — encrypt a *counter* to manufacture a keystream, then XOR. For block i ,

$$KS_i = E_k(\text{nonce} \parallel i), \quad C_i = P_i \oplus KS_i, \quad P_i = C_i \oplus KS_i,$$

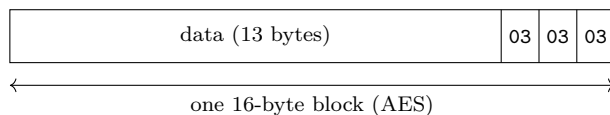
where $\text{nonce} \parallel i$ is the nonce concatenated with the block index i , forming a distinct b -bit input for every block, and KS_i is the resulting keystream block.



Trade-offs. The blocks are independent, so CTR is *fully parallel* and allows *random access* (jump straight to block i). It needs *no padding* (truncate the last keystream block to the message length), and uses only E_k — never the inverse D_k . Encryption and decryption are the *same* operation. The one rule, exactly as in §1: never reuse a (key, nonce) pair, or the keystream repeats and the “two-time pad” break applies.

2.4 Padding the last block

ECB and CBC need the plaintext to fill whole b -bit blocks; CTR and stream ciphers do not. The standard scheme, *PKCS#7*: if p bytes are missing from the final block, append p bytes each holding the *value* p . The value records how many to strip, so removal is unambiguous. If the message already fills the blocks exactly, append one whole extra block of padding — so there is always something to remove.



Here 13 data bytes leave $p = 3$ to fill a 16-byte block, so three 0x03 bytes are appended.

2.5 These modes hide, but do not protect

ECB, CBC, and CTR give *confidentiality* — they scramble the message — but no *integrity*: nothing detects tampering. CTR makes this stark. Since $P_i = C_i \oplus KS_i$, flipping any bit of C_i flips the *same* bit of the recovered plaintext, with no key needed:

$$(C_i \oplus \Delta) \oplus KS_i = P_i \oplus \Delta.$$

An attacker who knows the message format can thus make targeted edits the decryptor cannot notice. Confidentiality is not integrity. Real systems therefore use *authenticated encryption*, which bolts a message authentication code onto a mode (AES-GCM, ChaCha20-Poly1305) so any change is rejected — the subject of its own section.

2.6 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
E_k, D_k	block-cipher permutation and its inverse, on b -bit blocks
P_i, C_i	i -th b -bit plaintext / ciphertext block
n	number of blocks after padding
IV	initialization vector: random, non-secret, fresh per message
nonce	“number used once”; with i forms CTR’s per-block input
KS_i	i -th keystream block, $E_k(\text{nonce} \parallel i)$ (CTR)
$\oplus$	bitwise XOR
Δ	an attacker’s chosen bit-flip mask
i	block index, $1, \dots, n$